Robust Footstep Planning and LQR Control for Dynamic Quadrupedal Locomotion

Guiyang Xin, Songyan Xin, Oguzhan Cebe, Mathew Jose Pollayil, Franco Angelini, Manolo Garabini, Sethu Vijayakumar, and Michael Mistry

***Abstract*—In this letter, we aim to improve the robustness of dynamic quadrupedal locomotion through two aspects: 1) fast model predictive foothold planning, and 2) applying LQR to projected inverse dynamic control for robust motion tracking. In our proposed planning and control framework, foothold plans are updated at 400 Hz considering the current robot state and an LQR controller generates optimal feedback gains for motion tracking. The LQR optimal gain matrix with non-zero off-diagonal elements leverages the coupling of dynamics to compensate for system underactuation. Meanwhile, the projected inverse dynamic control complements the LQR to satisfy inequality constraints. In addition to these contributions, we show robustness of our control framework to unmodeled adaptive feet. Experiments on the quadruped ANYmal demonstrate the effectiveness of the proposed method for robust dynamic locomotion given external disturbances and environmental uncertainties.**

***Index Terms*—Legged robots, motion control, whole-body motion planning and control.**

I. Introduction

LEGGED robots have evolved quickly in recent years. Although there are robots, such as Spot from Boston Dynamics, which have been deployed in real industrial scenarios, researchers continue to explore novel techniques to improve locomotion performance. A popular technique is the staged approach which divides the larger problem into sub-problems and chains them together. Typically the pipeline is composed of state estimation, planning and control, which may be running at different frequencies. The motion planner typically runs at a slower frequency comparing to controller due to model nonlinearities and long planning horizons. The lower-level feedback controller runs at a higher frequency to resist model discrepancies and external disturbances. After years of evolution, optimization becomes the core approach for motion planning and control of legged robots.

Manuscript received October 15, 2020; accepted February 25, 2021. Date of publication March 24, 2021; date of current version April 12, 2021. This letter was recommended for publication by Associate Editor D. Pucci and Editor A. Kheddar upon evaluation of the reviewers' comments. This work was supported by Grants EPSRC UK RAI Hubs NCNR (EP/R02572X/1), ORCA (EP/R026173/1), EU Horizon 2020 THING (ICT-27-2017 780883), and NI (ICT-47-2020 101016970). *(Corresponding author: Songyan Xin.)*

Guiyang Xin, Songyan Xin, Oguzhan Cebe, and Michael Mistry are with the School of Informatics, Institute of Perception, Action, and Behaviour, University of Edinburgh, Edinburgh EH8 9AB, U.K. (e-mail: guiyang.xin@ed.ac.uk; sxin@ed.ac.uk; oguzhancebe@gmail.com; mmistry@inf.ed.ac.uk).

Mathew Jose Pollayil, Franco Angelini, and Manolo Garabini are with the Department of Information Engineering of the University of Pisa and with the Research Center E. Piaggio of the University of Pisa, 56122 Pisa, Italy (e-mail: mathewjosepollayil@gmail.com; frncangelini@gmail.com; manolo.garabini@gmail.com).

Sethu Vijayakumar is with the School of Informatics, Institute of Perception, Action, and Behaviour, University of Edinburgh, Edinburgh EH8 9AB, U.K., and also with the Shenzhen Institute for Artificial Intelligence and Robotics for Society (AIRS), (e-mail: sethu.vijayakumar@ed.ac.uk).

This letter has supplementary downloadable material available at https://doi.org/10.1109/LRA.2021.3068695, provided by the authors.

Digital Object Identifier 10.1109/LRA.2021.3068695

A. Related Planning Methods

Legged robot motion planning is a trade off between several criteria: formulation generality, model complexity, the planning horizon and computational efficiency. While the goal is to maximize all at once, this is not realistic given current available computational resources. As a result, different design choices lead to different formulations.

To generate motions in a more general and automated fashion, trajectory optimization (TO) has been used. In [1], a Zero Moment Point (ZMP)-based TO formulation is presented to optimize body motion, footholds and center of pressure simultaneously. It can generate different motion plans with multiple steps in less than a second. In a later work [2], a phase-based TO formulation is proposed to automatically determine the gait-sequence, step timings, footholds, body motion and contact forces. Motion for multiple steps can be still generated in few seconds. In these two works, the TO formulations are both extremely versatile in terms of motion types that can be generated, however, online Model Predictive Control (MPC) has not been demonstrated yet.

A linearized, single rigid-body model has been proposed [3] [4] to formulate the ground reaction force as a QP optimization problem and which can be solved in an MPC fashion. In both works, the footstep locations are provided by simple heuristics. Online TO based on a nonlinear single rigid-body model has been given in [5], and can generate stable dynamic motion for quadruped robots based on a given contact sequence. A whole-body dynamic model has been considered in [6][7] to generate robot motion in a MPC fashion. Crocoddyl [8] improves the computation speed further more. A frequency-aware MPC is proposed in [9] to deal with the bandwidth limitation problem for real hardware. In those four works, the contact planning problem has been decoupled from the whole-body motion planning problem.

Footstep optimization on biped robots has been proposed in [10] [11]. An underactuated linear inverted pendulum model (LIPM) has been used to formulate the footstep optimization problem. The idea has been extended and generalized to both biped and quadruped robots in our previous work [12]. In this

2377-3766 © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Johns Hopkins University. Downloaded on October 23,2026 at 01:55:55 UTC from IEEE Xplore. Restrictions apply.



Fig. 1. ANYmal with adaptive feet stepping on rough terrain.

work, we realize real-time footstep optimization that can be executed in a MPC fashion and test it on the hardware ANYmal.

B. Related Control Methods

In recent years, there has been a convergence among legged robot researchers to formulate the control problem as a Quadratic Program (QP) with constraints. The problem can be further decomposed into hierarchies to coordinate multiple tasks within whole-body control [13][14][15]. These optimization-based controllers usually rely on manually tuned diagonal feedback gain matrices. Also, these controllers only compute the best commands for the next control cycle, and therefore are not suitable for dynamic gaits with underactuation. Classical optimal control theory, such as LQR, can consider long or even infinite time horizons and generate optimal non-diagonial gain matrices exploiting dynamic coupling effects which benefit underactuated systems such as a cart-pole [16].

Classical LQR does not consider any constraints except system dynamics. However, for legged robots, we have to satisfy inequality constraints such as torque limits and friction cones on contact feet. The works [17][18] proposed to use the classical LQR controller for bipedal walking, but inequality constraints were not enforced. In this paper, we propose an LQR controller for dynamic gait control under the framework of projected inverse dynamics [19][20]. Projected inverse dynamic control enables us to control motion in a constraint-free subspace while satisfying inequality constraints in an orthogonal subspace. In our previous work, we used Cartesian impedance controllers within the constraint-free subspace to control both the base and swing legs during static walking of a quadruped robot. Here, we use LQR in the constraint-free subspace to replace the Cartesian impedance controller for base motion control and to handle the underactuation in the trotting gait.

C. Contributions

This paper focuses on improving the robustness of dynamic quadrupedal gaits. The trotting and pacing gaits of a quadruped robot will be studied and demonstrated in simulations and real experiments (see Fig. 1). The main contributions lie in the computation speed of the MPC and the optimal feedback control. As an additional contribution, our approach is shown to be valid both with the default spherical feet and the adaptive feet [21] with flexible soles. The main contributions are listed as follows:

1) We propose to formulate foothold planning as a QP problem subject to LIPM dynamics, which can be solved within the control cycle of $2.5ms$. Running re-planning at high frequency allows the robot to be responsive to disturbances and control commands. The higher the updating frequency of the MPC, the better the reactivity achieved by the robot.
2) We use unconstrained infinite-horizon LQR to generate optimal gains for base control in order to improve the robustness of the controller and cope with underactuation. Meanwhile, we inherit the advantage of our previous projected inverse dynamic framework to satisfy the inequality constraints in an orthogonal subspace, which is different to the purely QP-based controllers [13][14][15].

D. Paper Organization

The paper is organized in accordance with the hierarchical structure of the whole system, which is shown in Fig. 2. Given the desired velocity, the foothold planner plans future footsteps based on the current robot state which is explained in Section II. Section III describes the derivation of the LQR for base control. Simulations, experiments and discussions are given in Section IV. Finally, Section V draws the relevant conclusions.

II. Motion Generation

When considering dynamic gaits such as trotting, two contact points cannot constrain all six degrees of freedom (DOF) of the floating base. The system becomes underactuated as one DOF around the support line is not directly controlled. Researchers have been using the LIPM as an abstract model for balance control in this situation. The Centre of Mass (CoM) position and velocity can be predicted by solving the forward dynamics of the passive inverted pendulum. In order to keep long term balance, the next ZMP point has to be carefully selected to capture the falling CoM. For trotting, the ZMP point always lies on the support line formed by the supporting leg pair. As a result, the footholds optimization problem can be transformed to a ZMP optimization problem.

A. MPC Formulation

The dynamics of the linear inverted pendulum is as follows:

$$\begin{aligned} \ddot{x}_{CoM} &= \frac{g}{z_{CoM}}(x_{CoM} - p_x) \\ \ddot{y}_{CoM} &= \frac{g}{z_{CoM}}(y_{CoM} - p_y) \end{aligned} \tag{1}$$

where x_{CoM}, y_{CoM} and z_{CoM} are the CoM position coordinates, p_x and p_y are the coordinates of ZMP, g represents the acceleration of gravity. Considering z_{CoM} as constant, the dynamics become linear and result in the following solution:

$$\begin{aligned} \mathbf{x}_{CoM}(t) &= \mathbf{A}(t)\mathbf{x}^0_{CoM} + \mathbf{B}(t)p_x \\ \mathbf{y}_{CoM}(t) &= \mathbf{A}(t)\mathbf{y}^0_{CoM} + \mathbf{B}(t)p_y \end{aligned} \tag{2}$$

where $\mathbf{x}_{CoM} = [x_{CoM}\ \dot{x}_{CoM}]^\top$, $\mathbf{y}_{CoM} = [y_{CoM}\ \dot{y}_{CoM}]^\top$, are the state vectors, and $\mathbf{x}^0_{CoM}$ and $\mathbf{y}^0_{CoM}$ are the initial state

Authorized licensed use limited to: Johns Hopkins University. Downloaded on October 23,2026 at 01:55:55 UTC from IEEE Xplore. Restrictions apply.

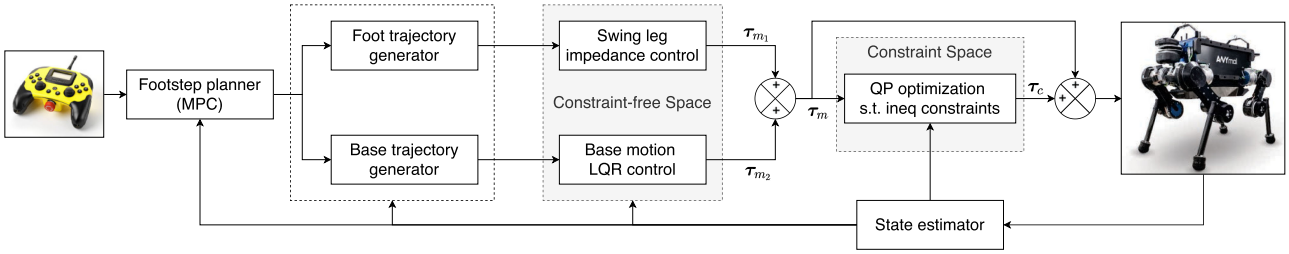


Fig. 2. Control framework block diagram. All the modules are running at 400 Hz. The joystick sends desired walking velocities. The MPC generates desired ZMPs. The ZMPs are mapped to foot placements which generate swing foot trajectories by interpolation. The desired base trajectory is generated based on desired ZMPs. An LQR and two impedance controllers are employed to track desired base trajectory and swing foot trajectories in the constraint-free space. Constraints, such as torque limits and friction cone, are satisfied in the constraint space.

vectors. $\mathbf{A}(t)$ and $\mathbf{B}(t)$ are defined as

$$\mathbf{A}(t) = \begin{bmatrix} \cosh(\omega t) & \omega^{-1}\sinh(\omega t) \\ \omega \sinh(\omega t) & \cosh(\omega t) \end{bmatrix} \tag{3}$$

$$\mathbf{B}(t) = \begin{bmatrix} 1 - \cosh(\omega t) \\ -\omega \sinh(\omega t) \end{bmatrix} \tag{4}$$

while $\omega = \sqrt{g/z_{CoM}}$.

For a periodic trotting gait with fixed swing duration T_s, assuming instant switching between single support phases, the states of N future steps along x direction can be predicted given step duration T_{s_i}

$$\begin{aligned} \mathbf{x}_{CoM_1} &= \mathbf{A}(T_{s_1})\mathbf{x}_{CoM_0} + \mathbf{B}(T_{s_1})p_{x_1} \\ \mathbf{x}_{CoM_2} &= \mathbf{A}(T_{s_2})\mathbf{x}_{CoM_1} + \mathbf{B}(T_{s_2})p_{x_2} \\ &\vdots \\ \mathbf{x}_{CoM_N} &= \mathbf{A}(T_{s_N})\mathbf{x}_{CoM_{N-1}} + \mathbf{B}(T_{s_N})p_{x_N} \end{aligned} \tag{5}$$

where $\mathbf{x}_{CoM_0}$ is the state at the moment of first touchdown, which can be computed from

$$\mathbf{x}_{CoM_0} = \mathbf{A}(t_0)\mathbf{x}^0_{CoM} + \mathbf{B}(t_0)p_{x_0} \tag{6}$$

where t_0 is the remaining period of the current swing phase. $\mathbf{x}^0_{CoM}$ and p_{x_0} are the current CoM state and ZMP location given by the state estimator which also runs at 400 Hz.

Also, considering the kinematic limits of the swing feet, the following inequality constraints are enforced:

$$\begin{bmatrix} p_{x_0} - d \\ -d \\ \vdots \\ -d \\ -d \end{bmatrix} \leq \begin{bmatrix} 1 & 0 & \cdots & 0 & 0 \\ -1 & 1 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & \cdots & 1 & 0 \\ 0 & 0 & \cdots & -1 & 1 \end{bmatrix} \begin{bmatrix} p_{x_1} \\ p_{x_2} \\ \vdots \\ p_{x_{N-1}} \\ p_{x_N} \end{bmatrix} \leq \begin{bmatrix} p_{x_0} + d \\ d \\ \vdots \\ d \\ d \end{bmatrix} \tag{7}$$

where d is a constant value derived from kinematic reachability relative to the stance feet. Additionally, Eq. (7) can also be used to avoid stepping into unfeasible pitches on the ground by redefining d.

The state along y direction has the same evolution as shown in Eq. (5). Regarding ZMPs as the system inputs, we define the cost function of the MPC as follows

$$\sum_{i=1}^{N} \frac{1}{2}[Q_i(\dot{x}_{CoM_i} - \dot{x}_{CoM_d})^2 + R_i(p_{x_i} - p_{x_{i-1}})^2] \tag{8}$$

where $\dot{x}_{CoM_d}$ is the desired CoM velocity in the x direction, Q_i and R_i are weight factors. The cost function for the y direction has the same form as Eq. (8). The MPC is formulated as a QP minimizing Eq. (8) subject to Eq. (5) and Eq. (7). Solving the QP results in the optimal ZMPs for the future N steps $\mathbf{p}^*_x = [p^*_{x_1}\ p^*_{x_2}\ \cdots\ p^*_{x_N}]^\top$.

Similarly, solving another QP for y direction yields the coordinate $\mathbf{p}^*_y = [p^*_{y_1}\ p^*_{y_2}\ \cdots\ p^*_{y_N}]^\top$ for the optimal ZMPs in this direction. It should be noted that the cost function for y direction is slightly different to Eq. (8), which is as follows

$$\sum_{i=1}^{N} \frac{1}{2}[Q_i(\dot{y}_{CoM_i} - \dot{y}_{CoM_d})^2 + R_i(p_{y_i} - p_{y_{i-1}} - s(-1)^i r_y)^2] \tag{9}$$

where r_y is a constant distance between right and left ZMPs. $r_y \neq 0$ for pacing gait to avoid self-collision while $r_y = 0$ for trotting gait. s indicates the support phase the robot is in, $s = 1$ for left support and $s = -1$ for right support.

We only use the first pair $\mathbf{p}^*_1 = (p^*_{x_1}\ \ p^*_{y_1})$ to generate the swing trajectory. Since the MPC is running in the same loop of controller, the position $\mathbf{p}^*_1 = (p^*_{x_1}\ \ p^*_{y_1})$ keeps updating during a swing phase given the updated CoM state $(\mathbf{x}^0_{CoM}$ \enskip $\mathbf{y}^0_{CoM})$ and desired CoM velocity $(\dot{x}_{CoM_d}$ \enskip $\dot{y}_{CoM_d})$.

B. Reference Trajectories of Trotting Gait

This section explains the algorithms to generate the desired trajectories of swing feet and the CoM for trotting gait based on the results of the MPC. The MPC provides the optimal ZMP that should be on the line connecting the next pair of support legs. We choose the ZMP to be the middle point of the support line for trotting gait. We keep the distance from the ZMP to each support foot location to be a fixed value r. Then we use the following equations to compute the desired footholds when the feet are

Authorized licensed use limited to: Johns Hopkins University. Downloaded on October 23,2026 at 01:55:55 UTC from IEEE Xplore. Restrictions apply.

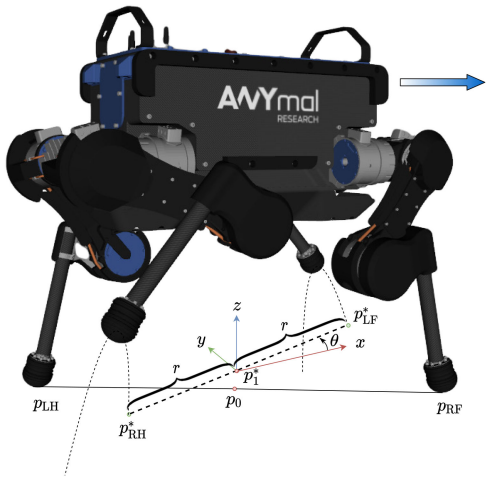


Fig. 3. Geometrical relationship between footholds and ZMPs. We assign the current ZMP (p_0) to be the middle point of the support line at the touchdown moment. The desired footholds (p^*_{LF}, p^*_{RH}) are calculated from the desired ZMP (p^*_1) and two foot pair parameters r and θ. r determines the distance between the foot pair and θ determines the orientation of the foot pair with respect to the robot heading direction. Nominal values are used for these two parameters. If there is a given steering command ω_z, the orientation can be updated $\theta = \theta_0 + \omega_z \cdot dt$.

swinging (in Fig. 3):

$$\begin{aligned}
\text{LF}: &\quad \begin{bmatrix} p_x^{\text{LF}} \\ p_y^{\text{LF}} \end{bmatrix} = \begin{bmatrix} p^*_{x_1} \\ p^*_{y_1} \end{bmatrix} + r \begin{bmatrix} \cos(\theta_0 + \Delta\theta) \\ \sin(\theta_0 + \Delta\theta) \end{bmatrix} \\
\text{RH}: &\quad \begin{bmatrix} p_x^{\text{RH}} \\ p_y^{\text{RH}} \end{bmatrix} = \begin{bmatrix} p^*_{x_1} \\ p^*_{y_1} \end{bmatrix} + r \begin{bmatrix} -\cos(\theta_0 + \Delta\theta) \\ -\sin(\theta_0 + \Delta\theta) \end{bmatrix} \\
\text{RF}: &\quad \begin{bmatrix} p_x^{\text{RF}} \\ p_y^{\text{RF}} \end{bmatrix} = \begin{bmatrix} p^*_{x_1} \\ p^*_{y_1} \end{bmatrix} + r \begin{bmatrix} \cos(\theta_0 - \Delta\theta) \\ -\sin(\theta_0 - \Delta\theta) \end{bmatrix} \\
\text{LH}: &\quad \begin{bmatrix} p_x^{\text{LH}} \\ p_y^{\text{LH}} \end{bmatrix} = \begin{bmatrix} p^*_{x_1} \\ p^*_{y_1} \end{bmatrix} + r \begin{bmatrix} -\cos(\theta_0 - \Delta\theta) \\ \sin(\theta_0 - \Delta\theta) \end{bmatrix}
\end{aligned} \tag{10}$$

where LF, RH, RF and LH are the abbreviations for left-fore, right-hind, right-fore and left-hind feet. θ_0 is a constant angle measured in the default configuration while $\Delta\theta$ is the rotation command sent by the users. For pacing gait, $\theta_0 = 0$.

Here we do not tackle the height changing issue. We use the current height of support feet to be the desired height of desired footholds for swing feet. The peak height during swing is a fixed relative offset. This technique has to be adapted for some tasks such as climbing stairs. However the robustness of the planner and controller can handle slightly rough terrains, as we demonstrate through experiments. After determining the desired footholds, we use cubic splines to interpolate the trajectories between the initial foot positions and desired footholds for the swing feet, and feed the one forward time step positions, velocities and accelerations to the controller.

The desired positions and velocities for CoM are determined by the LIPM, i.e., Eq. (2) where the initial states $\mathbf{x}^0_{CoM}$ and $\mathbf{y}^0_{CoM}$ are updating with 400 Hz as well. Setting the variable t in Eq. (2) to be a constant value $t = 2.5$ ms results in the desired CoM positions and velocities along x and y for controller. We set the desired height of CoM to be a constant value with respect to the average height of the support feet.

III. LQR for Base Control

We continue to use our projected inverse dynamic control framework [20] as it allows us to focus on designing trajectory tracking controllers without considering physical constraints. The physical constraints will be satisfied in an orthogonal subspace. This framework gives us the opportunity to use the classical LQR without any adaptation.

The dynamics of a legged robot can be projected into two orthogonal subspaces by using the projection matrix $\mathbf{P} = \mathbf{I} - \mathbf{J}_c^+ \mathbf{J}_c$ [22][23] as follows:

Constraint-free space:

$$\mathbf{PM}\ddot{\mathbf{q}} + \mathbf{Ph} = \mathbf{PS}\boldsymbol{\tau} \tag{11}$$

Constraint space:

$$(\mathbf{I} - \mathbf{P})(\mathbf{M}\ddot{\mathbf{q}} + \mathbf{h}) = (\mathbf{I} - \mathbf{P})\mathbf{S}\boldsymbol{\tau} + \mathbf{J}_c^\top \boldsymbol{\lambda}_c \tag{12}$$

where $\mathbf{q} = \begin{bmatrix} {}_I\mathbf{x}_b^\top & \mathbf{q}_j^\top \end{bmatrix}^\top \in SE(3) \times \mathbb{R}^n$, where ${}_I\mathbf{x}_b \in SE(3)$ denotes the floating base's position and orientation with respect to a fixed inertia frame I, meanwhile $\mathbf{q}_j \in \mathbb{R}^n$ denotes the vector of actuated joint positions. Also, we define the generalized velocity vector as $\dot{\mathbf{q}} = \begin{bmatrix} {}_I\mathbf{v}_b^\top & {}_B\boldsymbol{\omega}_b^\top & \dot{\mathbf{q}}_j^\top \end{bmatrix}^\top \in \mathbb{R}^{6+n}$, where ${}_I\mathbf{v}_b \in \mathbb{R}^3$ and ${}_B\boldsymbol{\omega}_b \in \mathbb{R}^3$ are the linear and angular velocities of the base with respect to the inertia frame expressed respectively in the I and B frame which is attached on the floating base. $\mathbf{M} \in \mathbb{R}^{(n+6)\times(n+6)}$ is the inertia matrix, $\mathbf{h} \in \mathbb{R}^{n+6}$ is the generalized vector containing Coriolis, centrifugal and gravitational effects, $\boldsymbol{\tau} \in \mathbb{R}^{n+6}$ is the vector of torques, $\mathbf{J}_c \in \mathbb{R}^{3k\times(n+6)}$ is the constraint Jacobian that describes $3k$ constraints, k denotes the number of contact points accounting foot contact and body contact, $\boldsymbol{\lambda}_c \in \mathbb{R}^{3k}$ are constraint forces acting on contact points, and

$$\mathbf{S} = \begin{bmatrix} \mathbf{0}_{6\times6} & \mathbf{0}_{6\times n} \\ \mathbf{0}_{n\times6} & \mathbf{I}_{n\times n} \end{bmatrix} \tag{13}$$

is the selection matrix with n dimensional identity matrix $\mathbf{I}_{n\times n}$.

Note that Eq. (11) together with Eq. (12) provides the whole system dynamics. The sum of the torque commands generated in the two subspaces will be the final command sent to the motors as shown in Fig. 2. In this paper, we focus on trajectory tracking control in the constraint-free subspace. We refer to our previous paper [20] for the inequality constraint satisfaction in the constraint subspace. The swing legs are controlled by impedance controllers proposed in our former paper [20]. In this paper, we propose to replace the impedance controller for base control with an LQR controller, benefiting from the optimal gain matrix instead of the hand-tuned diagonal impedance gain matrices.

The similar works of [17][18] did not enforce any inequality constraints with the classical LQR controller. The advantage of using projected inverse dynamics is that we can satisfy hard constraints, such as torque limits and friction cone constraints, in the constraint space by solving a QP as shown in Fig. 2, in case

Authorized licensed use limited to: Johns Hopkins University. Downloaded on October 23,2026 at 01:55:55 UTC from IEEE Xplore. Restrictions apply.

the LQR controller and impedance controller generate torque commands that violate those inequality constraints.

A. Linearization in Cartesian Space

Based on Eq. (11), we derive the forward dynamics

$$\ddot{\mathbf{q}} = \mathbf{M}_c^{-1}(-\mathbf{P}\mathbf{h} + \dot{\mathbf{P}}\dot{\mathbf{q}}) + \mathbf{M}_c^{-1}\mathbf{P}\mathbf{S}\boldsymbol{\tau} \tag{14}$$

where $\mathbf{M}_c = \mathbf{P}\mathbf{M} + \mathbf{I} - \mathbf{P}$ is called constraint inertia matrix [22]. Eq. (14) could be linearized with respect to the full state vector $\begin{bmatrix}\mathbf{q}^\top & \dot{\mathbf{q}}^\top\end{bmatrix}^\top$. However, the resulting linearized system would not be controllable as the corresponding controllability matrix is not full rank. Instead of resorting to one more projection as done in [17], we linearize the dynamics in the Cartesian space to control only the base states rather than all the states of a whole robot.

Just using a selection matrix, we can derive the forward dynamics with respect to ${}_I\mathbf{x}_b$

$$\ddot{\mathbf{x}}_b = \mathbf{J}_b\mathbf{M}_c^{-1}(-\mathbf{P}\mathbf{h} + \dot{\mathbf{P}}\dot{\mathbf{q}}) + \mathbf{J}_b\mathbf{M}_c^{-1}\mathbf{P}\mathbf{S}\boldsymbol{\tau} = f({}_I\mathbf{x}_b, \dot{\mathbf{x}}_b, \boldsymbol{\tau}) \tag{15}$$

where $\mathbf{J}_b = [\mathbf{I}_{6\times 6}\ \mathbf{0}_{6\times n}]_{6\times(n+6)}$, $\dot{\mathbf{x}}_b = \begin{bmatrix}{}_I\mathbf{v}_b^\top & {}_B\boldsymbol{\omega}_b^\top\end{bmatrix}^\top$.

By using Euler angles for the orientation in ${}_I\mathbf{x}_b$, we can define the state vector as

$$\mathbf{X} = \begin{bmatrix}{}_I\mathbf{x}_b \\ \dot{\mathbf{x}}_b\end{bmatrix}_{12\times 1} \tag{16}$$

We linearize Eq. (15) to state space dynamics around a configuration $(\mathbf{q}_0, \dot{\mathbf{q}}_0, \boldsymbol{\tau}_0)$ where $\boldsymbol{\tau}_0$ is the gravity compensation torques, yielding

$$\dot{\mathbf{X}} = \mathbf{A}_0^b\mathbf{X} + \mathbf{B}_0^b\boldsymbol{\tau} \tag{17}$$

Eq. (17) is detailed as

$$\begin{bmatrix}\dot{\mathbf{x}}_b \\ \ddot{\mathbf{x}}_b\end{bmatrix} = \begin{bmatrix}\mathbf{0} & \mathbf{I} \\ \mathbf{A}_{21} & \mathbf{A}_{22}\end{bmatrix}\begin{bmatrix}{}_I\mathbf{x}_b \\ \dot{\mathbf{x}}_b\end{bmatrix} + \begin{bmatrix}\mathbf{0} \\ \mathbf{B}_2\end{bmatrix}\boldsymbol{\tau} \tag{18}$$

where $\mathbf{A}_{21}$, $\mathbf{A}_{22}$ and $\mathbf{B}_2$ are defined as

$$\mathbf{A}_{21} = \frac{\partial f({}_I\mathbf{x}_b, \dot{\mathbf{x}}_b, \boldsymbol{\tau})}{\partial {}_I\mathbf{x}_b}|_{\mathbf{q}_0, \dot{\mathbf{q}}_0, \boldsymbol{\tau}_0} \tag{19}$$

$$\mathbf{A}_{22} = \frac{\partial f({}_I\mathbf{x}_b, \dot{\mathbf{x}}_b, \boldsymbol{\tau})}{\partial \dot{\mathbf{x}}_b}|_{\mathbf{q}_0, \dot{\mathbf{q}}_0} \tag{20}$$

$$\mathbf{B}_2 = \frac{\partial f({}_I\mathbf{x}_b, \dot{\mathbf{x}}_b, \boldsymbol{\tau})}{\partial \boldsymbol{\tau}} = \mathbf{J}_b\mathbf{M}_c^{-1}\mathbf{P}\mathbf{S}|_{\mathbf{q}_0} \tag{21}$$

For simplicity, we use a central finite difference method to compute the partial derivatives of Eq. (19) and Eq. (20). The deviation factor for finite difference we used for the experiments is 1×10^{-5}.

B. LQR Controller

We consider Eq. (17) as a linear time-invariant system and solve the infinite horizon LQR problem to compute the optimal feedback gain matrix $\mathbf{K}$. The cost function to be minimized is defined as

$$J = \int_0^\infty (\mathbf{X}^\top\mathbf{Q}\mathbf{X} + \boldsymbol{\tau}^\top\mathbf{R}\boldsymbol{\tau})dt \tag{22}$$

and the resulting controller for the base control is

$$\boldsymbol{\tau}_{m_2} = \mathbf{K}(\mathbf{X}_d - \mathbf{X}) + \boldsymbol{\tau}_d \tag{23}$$

where $\mathbf{X}_d$ is the desired state, $\boldsymbol{\tau}_d$ is the feedforward term derived from inverse dynamics based on the desired state.

We use ADRL Control Toolbox (CT) [24] to solve the infinite-horizon LQR problem and obtain the $\mathbf{K}$ matrix. It should be noted that the linearization is computed in every control cycle based on the current configuration $(\mathbf{q}_0, \dot{\mathbf{q}}_0, \boldsymbol{\tau}_0)$. The $\mathbf{K}$ matrix is updated at 400 Hz, which is different to [18] where they only compute the $\mathbf{K}$ matrices corresponding to few key configurations. We think linearization should be updated around current configuration in order to improve computation accuracy if the computation is fast enough.

In practice, we increase the weights in $\mathbf{R}$ of Eq. (22) for swing legs, relying more on the support legs for base control. Otherwise, the torque commands of Eq. (23) can affect the tracking of swing trajectories too much.

In addition, the motion planner in Section II feeds the desired CoM trajectory to the controllers, whereas the LQR controller controls the base pose. In theory, we should replace ${}_I\mathbf{x}_b$ with $\mathbf{x}_{CoM}$ in Eqs. (11)(12) and transform the dynamic equations to be with respect to CoM variables as in [25]. Then the LQR controller will directly track the desired CoM trajectory. In this paper, we approximately consider the translation of base along x and y aligned with CoM since the base dominates the mass of the whole robot.

IV. Validations

We use a torque controllable quadruped robot ANYmal made by ANYbotics to conduct our experiments. The onboard computer has an Intel 4th generation (HaswellULT) i7-4600U (1.4 GHz-2.1 GHz) processor and two HX316LS9IBK2/16 DDR3L memory cards. The robot weights approximately $35kg$ and has 12 joints actuated by Series Elastic Actuators (SEAs) with maximum torque of 40Nm. The real-time control cycle is $2.5ms$. The control software is developed based on Robot Operating System 1 (ROS 1). We use the dynamic modeling library Pinocchio [26] to perform the model linearization of Section III. An active set method based QP solver provided by ANYbotics is used to solve the QPs for the MPC planner and the controller. A video of the experimental results can be found at: https://youtu.be/khP6PQ9xuso.

A. Trotting Speed

We first tested the fastest walking speed when using the proposed algorithms. Figure 4 shows the recorded speeds along x direction in real robot experiment and in simulation. In simulation, the robot could stably trot forward with maximum speed 1.2 m/s. On real robot, the maximum speed reached 0.6 m/s. The results are reasonable since the trotting gait does not have a flying phase. The fact that the real robot cannot achieve as fast motion as in simulation is also reasonable considering model errors and other uncertainties. Model errors also cause drifting on the real robot which is difficult to resolve without external control loops. Constant values for the parameters of the gait planner were employed. They are $T_s = 0.3, z_{CoM} = 0.42, g = -9.8, N = 3$, $Q_i = 1000, R_i = 1, \theta_0 = 0.56, r = 0.41$. It should be noted that

Authorized licensed use limited to: Johns Hopkins University. Downloaded on October 23,2026 at 01:55:55 UTC from IEEE Xplore. Restrictions apply.

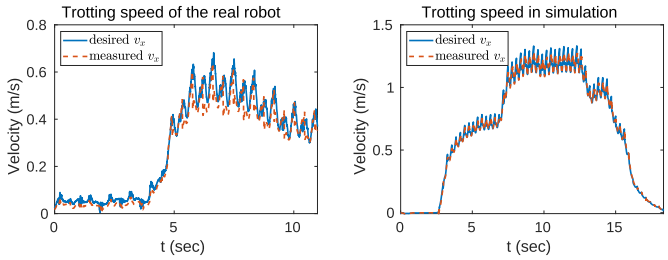


Fig. 4. Recorded fastest trotting speeds on real robot and in simulation. The desired velocities are generated from LIPM dynamics, i.e. Eq. (2), which explains why the reference velocities are not smooth.

the prediction number N in the MPC does not need to be as large as possible with the concern of computation efficiency. We tested $N = 2 \sim 5$, and they showed similar performance.

B. Push Recovery

In this subsection, we demonstrate the benefit of high frequency replanning for disturbance rejection. We first use simulation to show the replanned footholds and trajectories as shown in Fig. 5. The disturbance is added when RF and LH feet are swinging. The disturbance results in sharp state changes. The MPC computed the new footholds after receiving the updated state. Figure 6 presents snapshot photos of the push-recovery experiment on the real robot during trotting while recorded state data is shown in Fig. 7. The robot was kicked four times roughly along the y direction. We can see the peak velocity of y reached $-1m/s$ during the last two kicks, but it was quickly regulated back to normal using one or two steps. The orientation did not change too much after kicking, which also indicates the robustness of the method.

C. Balance Control

Most of the trotting gait control algorithms rely on quick switching of swing and stance phases to achieve dynamic balance. Recently researchers demonstrated that quadruped robots with point feet can stand on two feet to maintain balance [27][28]. Although we did not manage balancing on two feet on our robot, we compared the longest period of swing phase of trotting when using our proposed LQR controller versus the default trotting controller of ANYmal [29]. The longest swing phase when using LQR is $0.63s$ whereas the default controller only achieved $0.42s$. When the base is controlled by our previous impedance controller, the longest swing phase is $0.4s$. This verifies the improved performance of our LQR controller in terms of balance control. Figure 8 shows two gain matrices for the two phases of trotting during this experiment. It should be noted that the gain matrices are updated in every control cycle (but the changes are small). We can see that the elements of the first 6 rows are 0 because of the existence of selection matrix $\mathbf{S}$. The $\mathbf{Q}_{12\times12}$ we used in the experiment was $\mathbf{Q} = \text{diag}(\text{diag}(1500)_{6\times6},\ \text{diag}(1)_{6\times6})$. The $\mathbf{R}_{18\times18}$ was switched depending on the phase. The diagonal elements corresponding to the two swing legs in $\mathbf{R}$ are 10 times larger than the other diagonal elements for stance legs and the base, reducing the efforts of swing legs in balance control. The elements in $\mathbf{R}$ for stance legs and the base we used in this experiment are 0.03.

D. Trotting on Slippery Terrains

The most important advantage of the proposed control framework compared to similar works [17][18] is that we can satisfy inequality constraints while using LQR for trajectory tracking. We do not change the classical LQR to be a constrained LQR. By contrast, using the projected inverse dynamic control allows us to satisfy inequality constraints in the constraint subspace. The LQR controller only serves as a trajectory tracking controller and does not need to consider the inequality constraints. The QP optimization in the constraint subspace plays the role of trading off different constraints. For example, as we have shown in our previous paper [20] for static gait, trajectory tracking performance will be sacrificed to prevent slipping if torque commands for trajectory tracking generate contact forces beyond the friction cones. Here we demonstrate that our proposed controller can satisfy friction cone constraints for dynamic gaits as well. Figure 9 shows the controller can keep the contact forces within the friction cone after reducing the friction coefficient to match the actual friction coefficient of the terrain. The smallest friction coefficient we achieved in simulation for trotting in spot on flat terrain is 0.07. However it is difficult to trot on such slippery terrain because the trajectory tracking is quite poor in this situation.

E. Transition from Trotting to Pacing

Pacing gait is a more dynamic gait compared to trotting since the CoM is always off the supporting line. The difference between trotting and pacing in terms of the MPC formulation is that there will be a constant offset r_y between p_{y_i} and $p_{y_{i-1}}$ (see Fig. 10) in the cost function Eq. (9) in order to avoid conflicts of the right and left feet. In our experiment, we specified a transition motion of shifting the base to a side to start pacing. We can also remove this transition motion by reducing the gait period or reducing the distance between left and right feet. The gait period in this experiment was $0.44s$ with $r_y = 0.08m$. On the controller side, we used the same $\mathbf{Q}$ and $\mathbf{R}$ for trotting and pacing.

F. Outdoor Test With Adaptive Feet

In this subsection, we test the versatility of our approach with adaptive feet SoftFoot-Q [21] in outdoor environments. Figure 11 shows a typical case of the adaptive feature. Compared to the traditional sphere feet, the adaptive feet have larger contact surface. Those features will benefit the traversability of rough terrains with rocks, loose gravel and rubble by enlarging the contact surfaces with ground. We performed experiments in trotting locomotion on rough terrains outside our lab as shown in Fig. 12. It should be noted that our controller did not take the two DOF (one DOF less than the case with a spherical foot) passive ankle into account. The model errors caused by the adaptive feet were treated as disturbances by the controller, where the success of the tests shows the robustness of our controller.

Authorized licensed use limited to: Johns Hopkins University. Downloaded on October 23,2026 at 01:55:55 UTC from IEEE Xplore. Restrictions apply.

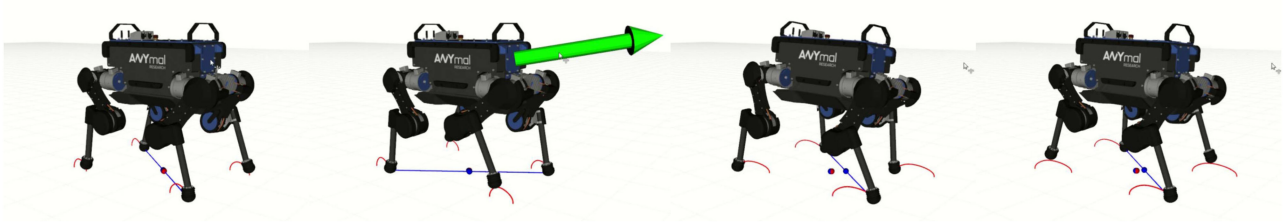


Fig. 5. Footstep and swing trajectory replanning under disturbances. The robot is walking forward and an external force (green arrow) is applied to the base of the robot. The push results in a sudden change of the CoM position and velocity. The footstep planner uses the updated state to replan the footholds. The swing trajectories (red lines) are updated accordingly.

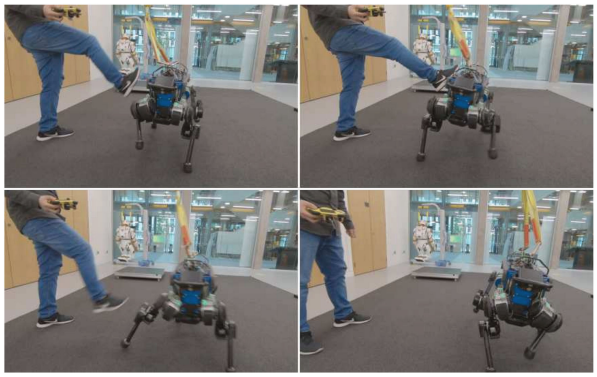


Fig. 6. Kicking the robot during trotting. Benefiting from the 400 Hz MPC update frequency, the robot can quickly update the optimal footholds to recover from disturbances.

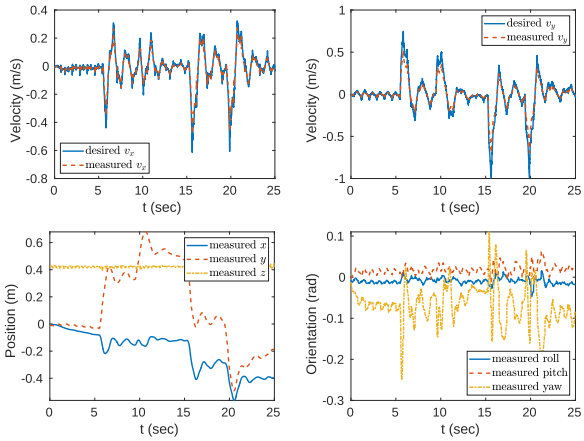


Fig. 7. Recorded state when the robot was kicked four times. The desired Euler angles were 0. The robot was quickly regulated back to normal even though the velocity reached $-1m/s$ after kicking.

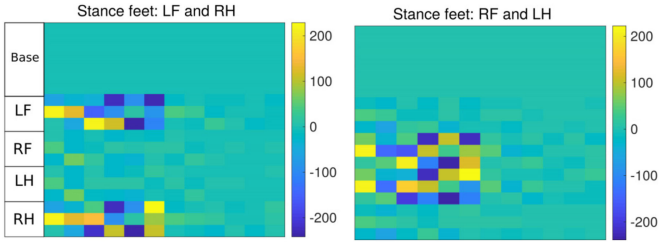


Fig. 8. Visualization of the gain matrix as computed by the LQR controller during trotting. The size of the gain matrix is 18 by 12. The first 6 columns correspond to the position and orientation while the remaining 6 columns are for velocity control. Off-diagonal gains demonstrate that dynamic coupling effects may be exploited for control.

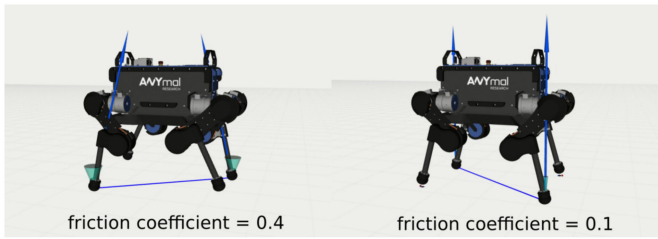


Fig. 9. The friction cone constraints are satisfied by the controller. The blue arrows represent the actual contact forces while the green cones denote the friction cones.

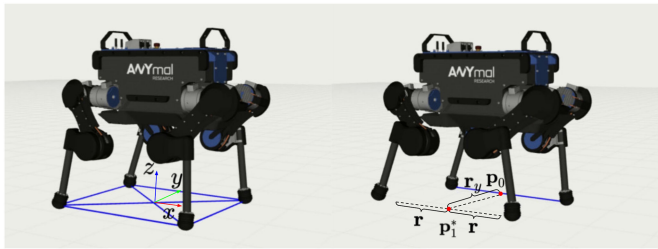


Fig. 10. A base shifting motion is needed to transit from trotting to pacing.



Fig. 11. The SoftFoot-Q, an adaptive foot for quadrupeds. θ_1 and θ_2 indicate the passive joints of the ankle.

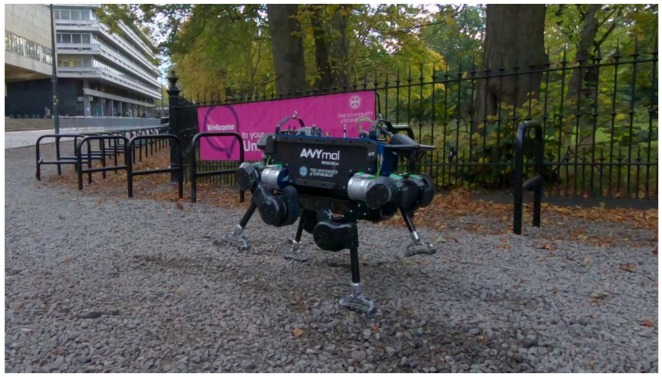


Fig. 12. Trotting out of the lab with adaptive feet on rubble terrain.

Authorized licensed use limited to: Johns Hopkins University. Downloaded on October 23,2026 at 01:55:55 UTC from IEEE Xplore. Restrictions apply.

V. Conclusion

This letter presents a full control framework for dynamic gaits where all the modules are running with the same frequency. The robustness of the dynamic walking is improved significantly by two factors. The first factor is the MPC planner, which mostly contributes to rejecting large disturbances, such as kicking the robot, because the MPC uses footsteps to regulate the state of the robot. The second factor is the LQR controller for balancing control, which also undertakes the duty of trajectory tracking. The method is general and shown to able to work both with spherical and adaptive feet. The latter were seen to reduce the slipping chance on rough terrains. The outdoor experiments demonstrate the robustness of locomotion after adopting the proposed methods and assembling the adaptive feet.

Future work will focus on adapting the current planner to consider terrain information to handle large slopes and stairs. Also, the new feet can be used to measure the local inclination of the ground which can improve the accuracy of the terrain information, similar to [30].

Acknowledgment

The authors would like to thank Dr. Quentin Rouxel and Dr. Carlos Mastalli for introduction on using the Pinocchio rigid body dynamics library. The authors also would like to thank the editor and reviewers for their useful comments.

References

[1] A. W. Winkler, F. Farshidian, D. Pardo, M. Neunert, and J. Buchli, "Fast trajectory optimization for legged robots using vertex-based ZMP constraints," *IEEE Robot. Autom. Lett*., vol. 2, no. 4, pp. 2201–2208, Oct. 2017.

[2] A. W. Winkler, C. D. Bellicoso, M. Hutter, and J. Buchli, "Gait and trajectory optimization for legged systems through phase-based end-effector parameterization," *IEEE Robot. Autom. Lett*., vol. 3, no. 3, pp. 1560–1567, Jul. 2018.

[3] J. Di Carlo, P. M. Wensing, B. Katz, G. Bledt, and S. Kim, "Dynamic locomotion in the mit cheetah 3 through convex model-predictive control," in *Proc. IEEE Int. Conf. Intell. Robots Syst*., 2018, pp. 1–9.

[4] D. Kim, J. Di Carlo, B. Katz, G. Bledt, and S. Kim, "Highly dynamic quadruped locomotion via whole-body impulse control and model predictive control," 2019, *arXiv:1909.06586*.

[5] O. Cebe, C. Tiseo, G. Xin, H.-c. Lin, J. Smith, and M. Mistry, "Online dynamic trajectory optimization and control for a quadrupedrobot," 2020, *arXiv:2008.12687*.

[6] J. Koenemann *et al.*, "Whole-body model-predictive control applied to the hrp-2 humanoid," in *Proc. IEEE Int. Conf. Intell. Robots Syst*., 2015, pp. 3346–3351.

[7] M. Bjelonic, R. Grandia, O. Harley, C. Galliard, S. Zimmermann, and M. Hutter, "Whole-body mpc and online gait sequence generation for wheeled-legged robots," 2020, *arXiv:2010.06322*.

[8] C. Mastalli *et al.*, "Crocoddyl: An efficient and versatile framework for multi-contact optimal control," in *Proc. IEEE Int. Conf. Robot. Autom*., 2020, pp. 2536–2542.

[9] R. Grandia, F. Farshidian, R. Ranftl, and M. Hutter, "Feedback mpc for torque-controlled legged robots," in *Proc. IEEE Int. Conf. Intell. Robots Syst*., 2019, pp. 4730–4737.

[10] S. Faraji, S. Pouya, C. G. Atkeson, and A. J. Ijspeert, "Versatile and robust 3d walking with a simulated humanoid robot (atlas): A. model predictive control approach," in. *Proc IEEE Int. Conf. Robot. Autom*., 2014, pp. 1943–1950.

[11] S. Feng, X. Xinjilefu, C. G. Atkeson, and J. Kim, "Robust dynamic walking using online foot step optimization," in *Proc. IEEE Int. Conf. Intell. Robots Syst*., 2016, pp. 5373–5378.

[12] S. Xin, R. Orsolino, and N. G. Tsagarakis, "Online relative footstep optimization for legged robots dynamic walking using discrete-time model predictive control," in *Proc. IEEE Int. Conf. Intell. Robots Syst*., 2019, pp. 513–520.

[13] L. Saab, O. E. Ramos, F. Keith, N. Mansard, P. Soueres, and J. Y. Fourquet, "Dynamic whole-body motion generation under rigid contacts and other unilateral constraints," *IEEE Trans. Robot*., vol. 29, no. 2, pp. 346–362, Apr. 2013.

[14] A. Del Prete, F. Nori, G. Metta, and L. Natale, "Prioritized motion-force control of constrained fully-actuated robots:task space inverse dynamics," *Robot. Auton. Syst.*, vol. 63, pp. 150–157, 2015.

[15] A. Herzog, N. Rotella, S. Mason, F. Grimminger, S. Schaal, and L. Righetti, "Momentum control with hierarchical inverse dynamics on a torque-controlled humanoid," *Auton. Robots*, vol. 40, no. 3, pp. 473–491, 2016.

[16] P. Reist and R. Tedrake, "Simulation-based lqr-trees with input and state constraints," in. *Proc IEEE Int. Conf. Robot. Autom*., 2010, pp. 5504–5510.

[17] S. Mason, L. Righetti, and S. Schaal, "Full dynamics lqr control of a humanoid robot: An experimental study on balancing and squatting," in *Proc. IEEE Int. Conf. Humanoid Robots*, 2014, pp. 374–379.

[18] S. Mason, N. Rotella, S. Schaal, and L. Righetti, "Balancing and walking using full dynamics lqr control with contact constraints," in *Proc. IEEE Int. Conf. Humanoid Robots*, 2016, pp. 63–68.

[19] G. Xin, H.-C. Lin, J. Smith, O. Cebe, and M. Mistry, "A model-based hierarchical controller for legged systems subject to external disturbances," in. *Proc IEEE Int. Conf. Robot. Autom*., 2018, pp. 4375–4382.

[20] G. Xin, W. Wolfslag, H.-C. Lin, C. Tiseo, and M. Mistry, "An optimization-based locomotion controller for quadruped robots leveraging cartesian impedance control," *Front. Robot. AI*, vol. 7, 2020, Art. no. 48.

[21] M. G. Catalano *et al.*, "Adaptive Feet for Quadrupedal Walkers," *IEEE Trans. Robot*., 2020, [*Under Review*]. [Online]. Available: https://www.dropbox.com/s/84ry3we72c7kt6s/adaptive_feet_preprint.pdf

[22] F. Aghili, "A unified approach for inverse and direct dynamics of constrained multibody systems based on linear projection operator: Applications to control and simulation," *IEEE Trans. Robot*., vol. 21, no. 5, pp. 834–849, Oct. 2005.

[23] M. Mistry and L. Righetti, "Operational space control of constrained and underactuated systems," in *Proc. Robot.: Sci. Syst.*, 2011. [Online]. Available: https://research.birmingham.ac.uk/portal/en/publications/operational-space-control-of-constrained-and-underactuated-systems(f9709495-978b-4426-b78d-4ee6d0be4b37)/export.html

[24] M. Giftthaler, M. Neunert, M. Stüuble, and J. Buchli, "The control toolbox - an open-source c library for robotics, optimal and model predictive control," in *Proc. IEEE Int. Conf. Simulation, Modeling, Program. Auton. Robots*, 2018, pp. 123–129.

[25] B. Henze, M. A. Roa, and C. Ott, "Passivity-based whole-body balancing for torque-controlled humanoid robots in multi-contact scenarios," *Int. J. Robot. Res*., vol. 35, no. 12, pp. 1522–1543, 2016.

[26] J. Carpentier *et al.*, "The pinocchio c library - a fast and flexible implementation of rigid body dynamics algorithms and their analytical derivatives," in *Proc. IEEE Int. Symp. Syst. Integrations*, 2019, pp. 614–619.

[27] M. Chignoli and P. M. Wensing, "Variational-based optimal control of underactuated balancing for dynamic quadrupeds," *IEEE Access*, vol. 8, pp. 49 785–49797, 2020.

[28] C. Gonzalez, V. Barasuol, M. Frigerio, R. Featherstone, D. G. Caldwell, and C. Semini, "Line walking and balancing for legged robots with point feet," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Las Vegas, NV, USA, 2020, pp. 3649–3656, doi: 10.1109/IROS45743.2020.9341743.

[29] C. Gehring, S. Coros, M. Hutter, M. Bloesch, M. A. Hoepflinger, and R. Siegwart, "Control of dynamic gaits for a quadrupedal robot," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2013, pp. 3287–3292.

[30] G. Valsecchi, R. Grandia, and M. Hutter, "Quadrupedal locomotion on uneven terrain with sensorized feet," *IEEE Robot. Autom. Lett*., vol. 5, no. 2, pp. 1548–1555, Apr. 2020.

Authorized licensed use limited to: Johns Hopkins University. Downloaded on October 23,2026 at 01:55:55 UTC from IEEE Xplore. Restrictions apply.